AIA11: Explainable AI (XAI)

Module Introduction and Theoretical Foundations

Explainable AI (XAI) bridges the **interpretability-explanation-scrutability gap**, enabling trust, debugging, and regulatory compliance for black-box models. This module covers **local** (SHAP, LIME), **global** (feature importance), and **intrinsic** (attention-based) methods with mathematical foundations and production implementations.

XAI Taxonomy (Molnar 2020):

text

Intrinsic vs Post-hoc

Local vs Global

Model-specific vs Model-agnostic

Theoretical Foundations:

1. **Shapley Values**: Game theory for feature attribution
2. **LIME**: Local linear approximation
3. **PDP/ICE**: Marginal feature effects
4. **Counterfactuals**: Minimal changes for outcome flip

(Figure 1: XAI method landscape)

Section 1: SHAP - Shapley Additive Explanations

Shapley Value Theory (Shapley 1953):

text

$\phi_i = \Sigma_{S\subseteq N\setminus\{i\}} |S|! (M-|S|-1)! / M! [v(S\cup\{i\}) - v(S)]$

Average marginal contribution across coalitions

SHAP Approximation Methods:

1. **Kernel SHAP**: Weighted linear regression
2. **Tree SHAP**: Exact for trees ($O(TL^2)$)
3. **Deep SHAP**: Deep Taylor decomposition

Complete SHAP Implementation:

python

```python
import shap
import xgboost as xgb
from shap import TreeExplainer, DeepExplainer
```

```
class SHAPExplainer:
    def __init__(self, model, background_data=None):
        self.model = model
        if isinstance(model, xgb.Booster):
            self.explainer = TreeExplainer(model)
        else:
            self.explainer = DeepExplainer(model, background_data)

    def explain_instance(self, instance, labels=None):
        shap_values = self.explainer.shap_values(instance)

        if isinstance(shap_values, list):
            # Multi-class
            return {label: shap_values[i] for i, label in enumerate(labels)}
        return shap_values

    def shap_summary_plot(self, shap_values, X, feature_names):
        shap.summary_plot(shap_values, X, feature_names=feature_names)

    def shap_force_plot(self, shap_values, instance, feature_names):
        shap.force_plot(self.explainer.expected_value,
                        shap_values, instance,
                        feature_names=feature_names)

# Production usage
model = xgb.XGBClassifier().fit(X_train, y_train)
explainer = SHAPExplainer(model)
shap_values = explainer.explain_instance(X_test[0])
explainer.shap_force_plot(shap_values, X_test[0], feature_names)
```

SHAP Properties:

text

Efficiency: $\Sigma\ \phi_i(f) = f(x)$

Symmetry: Equal contribution → equal ϕ

Additivity: $\phi(f+g) = \phi(f) + \phi(g)$

Section 2: LIME - Local Interpretable Model-agnostic Explanations

LIME Algorithm (Ribeiro 2016):

text

1. Perturb instance x → neighborhood Z
2. Get predictions f(z) ∀ z ∈ Z
3. Learn g ∈ G (interpretable) minimizing $L(f,g,\pi_x)$

Mathematical Formulation:

text

$$L(f,g,\pi_x) = \Sigma_{z,z'\in Z}\ \pi_x(z)\ (f(z) - g(z'))^2 + \Omega(g)$$

LIME Implementation:

python

```python
from lime.lime_tabular import LimeTabularExplainer


class LIMEExplainer:
    def __init__(self, model, training_data, feature_names, categorical_features=[]):
        self.explainer = LimeTabularExplainer(
            training_data,
            feature_names=feature_names,
            categorical_features=categorical_features,
            mode='classification'
        )
        self.model = model

    def explain_instance(self, instance, num_features=10, num_samples=5000):
        explanation = self.explainer.explain_instance(
            instance, self.model.predict_proba,
```

```
        num_features=num_features, num_samples=num_samples
    )
    return explanation

  def plot_explanation(self, explanation):
    explanation.show_in_notebook()

# Usage
lime_explainer = LIMEExplainer(model, X_train, feature_names)
exp = lime_explainer.explain_instance(X_test[0])
lime_explainer.plot_explanation(exp)
```

Section 3: Global Explanations - PDP, ICE, Feature Importance

Partial Dependence Plot (PDP):

text

PDP_f,S(x_S) = E_{X_C} [f(X_S, X_C)]

Average prediction fixing feature set S

Individual Conditional Expectation (ICE):

text

ICE curves for each instance

Implementation:

python

```
from sklearn.inspection import partial_dependence, PartialDependenceDisplay

def global_explain(model, X, features):
  # PDP for top 3 features
  pdp_results = partial_dependence(
    model, X, features=[0,1,2],
    grid_resolution=50, kind='average'
  )
```

```python
    # Plot
    PartialDependenceDisplay.from_estimator(
        model, X, features=[0,1,2]
    )

    # Feature importance (permutation)
    from sklearn.inspection import permutation_importance
    result = permutation_importance(model, X_test, y_test,
                                    n_repeats=10, random_state=42)

    return pdp_results, result.importances_mean

pdp, importances = global_explain(model, X_train, feature_names)
```

Section 4: Attention-Based Explanations for Transformers

Attention Flow (Abnar 2020):

text

AttFlow_i,j = Σ_k Att_k[i,j] * (1 - |h_k[i] - h_k[j]|)

Propagate attention through layers

Attention Rollout:

text

Rollout_{l,i,j} = Σ_k Att_l[i,k] Rollout_{l+1,k,j}

Transformer Explainer:

python

```python
def attention_rollout(model, input_ids, layer_agg='mean'):
    """
    Compute attention rollout for transformer
    """
    embeddings = model.transformer.wte(input_ids)
    attention_maps = []
```

```
    for layer in model.transformer.h:
        layer_attention = layer.attn.c_attn(embeddings)
        attention_maps.append(layer_attention)

    # Rollout aggregation
    rollout = torch.eye(input_ids.shape[1])
    for attn in reversed(attention_maps):
        rollout = torch.matmul(rollout, attn)

    return rollout

def visualize_attention(model, tokens):
    rollout = attention_rollout(model, tokens)
    plt.imshow(rollout.detach().numpy(), cmap='hot')
    plt.xticks(range(len(tokens)), tokens)
    plt.yticks(range(len(tokens)), tokens)
    plt.title('Attention Rollout')
```

Section 5: Counterfactual and Causal Explanations

Counterfactuals (Wachter 2017):

text

Find minimal perturbation δ: f(x+δ) = y_target, minimize ||δ||

Causal Impact:

text

do-calculus for interventions

Structural Causal Models

Counterfactual Generation:

python

```
from alibi.explainers import CounterFactualProto

cf_proto = CounterFactualProto(
```

```
    predictor,
    shape=(1,784),  # MNIST
    target_class='other',
    max_iterations=1000
)

explanation = cf_proto.explain(X_test[0:1])
cf_proto.plot(explanation)
```

Section 6: Production XAI Systems

XAI Serving Architecture:

text

Model → Feature extractor → Explainer → Visualization

Async explanation generation

Compliance Framework (EU AI Act):

text

High-risk systems: Risk assessment + logging

General purpose: Transparency reporting

Monitoring Dashboard:

python

```python
class XAIServer:
    def __init__(self, model, explainers):
        self.model = model
        self.explainers = explainers  # {'shap': SHAPExplainer, ...}

    @app.route('/predict_explain', methods=['POST'])
    def predict_explain(self):
        data = request.json['data']
        prediction = self.model.predict(data)

        explanations = {}
```

```
    for name, explainer in self.explainers.items():
        explanations[name] = explainer.explain_instance(data)

    return {
        'prediction': prediction,
        'explanations': explanations,
        'trust_score': self.compute_trust(explanations)
    }
```

Section 7: Case Studies and Evaluation

Case Studies:

1. **Healthcare**: SHAP for treatment recommendations
2. **Finance**: Counterfactuals for loan denials
3. **LLMs**: Attention for faithfulness

XAI Evaluation:

text

1. Faithfulness: Does explanation match model behavior?

2. Plausibility: Human interpretability

3. Stability: Consistent perturbations → consistent explanations

Capstone: Production XAI Platform

Multi-model explanations

Interactive dashboards

Compliance reporting